

Ejercicio 4

Simulador de cola de transacciones de un robot

Presentado por:

Danny Heredia

1. Entendimiento del problema

El ejercicio consiste en hacer un programa en Python que simule una cola de transacciones. El programa debe revisar cada transacción una por una para saber si se puede procesar o si tiene algún problema.

Cada transacción tiene un número de identificación, un cliente, un monto y un estado. Si hay un problema con los datos, la transacción falla. Si el problema es por falta de conexión, se debe intentar nuevamente hasta un máximo de tres intentos.

Al final, el programa debe mostrar los intentos realizados y un resumen de los resultados.

2. Diseño de la solución

Pseudocódigo del proceso

INICIO

Crear una lista con las 10 transacciones.

Recorrer las transacciones una por una.

Revisar el monto y el cliente.

Si el monto es menor o igual a 0 o el cliente está vacío,
registrar una falla de negocio y no realizar reintentos.

Si el estado es SIN_CONEXION,
realizar hasta 3 intentos.

Si el ID es par, la transacción se recupera en el segundo intento.

Si el ID es impar, la transacción falla después de 3 intentos.

Si el estado es OK, la transacción es exitosa en el primer intento.

Contar las transacciones exitosas y fallidas.

Calcular el porcentaje de éxito.

Mostrar el resumen final.

FIN

3. Estructuras de datos que utilicé

En el programa utilicé principalmente una lista para guardar las transacciones y diccionarios para almacenar los datos de cada transacción.

Lista: contiene las 10 transacciones y permite recorrerlas una por una en el orden en que fueron agregadas.

Diccionario: cada transacción se guarda con sus datos: id, cliente, monto y estado_origen.

Variables: se utilizan para llevar el conteo de transacciones exitosas, fallidas y reintentos.

Elegí estas estructuras porque son sencillas y permiten organizar los datos de las transacciones de una forma fácil de consultar y procesar.

En el programa se utilizaron tres funciones principales:

revisar_transaccion(transaccion)

4. Descripción de las funciones

Recibe una transacción y revisa si el monto es válido y si el cliente tiene información. Devuelve una falla de negocio cuando alguno de estos datos no es correcto o devuelve OK cuando los datos están bien.

procesar_transaccion(transaccion)

Recibe una transacción y determina qué resultado debe tener. Si tiene una falla de negocio, la registra sin reintentar. Si está en estado OK, la marca como exitosa. Si tiene SIN_CONEXION, indica que necesita realizar intentos.

hacer_intentos(transaccion)

Recibe una transacción con problemas de conexión y realiza los intentos hasta un máximo de tres. Si el ID es par, se recupera en el segundo intento. Si el ID es impar, llega al tercer intento y queda como falla de sistema.

Esta descripción corresponde a las funciones que realmente utilizamos en el código.

5. Reglas de negocio implementadas

1. Falla por datos incorrectos: si el monto es menor o igual a 0, o el cliente está vacío, la transacción se marca como FALLA_NEGOCIO y no se vuelve a intentar.

2. Falla por falta de conexión: si el estado es SIN_CONEXION, se realizan reintentos hasta un máximo de 3 intentos.

3. Transacción con ID par: cuando una transacción con SIN_CONEXION tiene un ID par, se recupera en el segundo intento y queda como EXITOSA.

4. Transacción con ID impar: cuando una transacción con SIN_CONEXION tiene un ID impar, realiza los 3 intentos y finalmente queda como FALLA_SISTEMA.

5. Transacción sin problemas: cuando el estado es OK y los datos son válidos, la transacción queda como EXITOSA en el primer intento.
6. Registro de intentos: cada intento muestra el ID, el número de intento y el resultado.

Estas reglas son las que se aplicaron en el programa para procesar las transacciones y obtener el resumen final.

6. Casos borde considerados

Durante el desarrollo se tuvieron en cuenta algunas situaciones que podían generar errores en el proceso.

- * Monto igual a 0: se consideró como una falla de negocio y la transacción no se vuelve a intentar.
- * Cliente vacío: se considera una falla de negocio y no se realizan reintentos.
- * Falta de conexión: cuando una transacción tiene SIN_CONEXION, se realizan los intentos correspondientes.
- * Máximo de intentos: se estableció un máximo de 3 intentos para evitar que una transacción se repita indefinidamente.
- * ID par e impar: se utilizaron para simular los diferentes resultados de una transacción con problemas de conexión.
- * Lista de transacciones: se trabajó con una lista de 10 transacciones para comprobar los diferentes casos del ejercicio.

7. Instrucciones de ejecución

Para ejecutar el programa se deben seguir estos pasos:

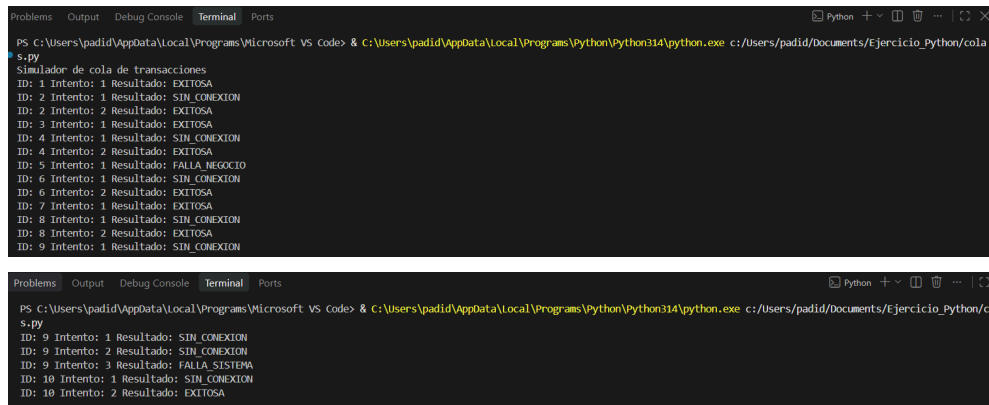
1. Abrir el proyecto donde se encuentra el archivo de Python.
2. Abrir el archivo .py del ejercicio.
3. Verificar que las transacciones estén creadas correctamente.
4. Ejecutar el programa desde el entorno de Python.
5. Esperar a que se procesen las transacciones una por una.
6. Revisar en la terminal los intentos realizados y sus resultados.
7. Al finalizar, revisar el resumen con el total de transacciones, resultados, reintentos y porcentaje de éxito.

8. Evidencia

En esta sección se muestran las capturas de pantalla de la ejecución del programa.

Las capturas permiten observar el procesamiento de las transacciones, los intentos realizados cuando existe una falta de conexión y el resumen final de los resultados.

1. Ejecución de las transacciones y registro de los intentos.



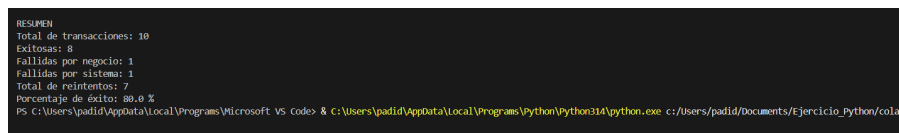
The first screenshot shows the output of a Python script named 'cola.py'. It simulates a queue of transactions with 10 IDs. For each ID, it shows the attempt number and the result. The results are: ID 1 (1 attempt, EXITOSA), ID 2 (1 attempt, SIN_CONEXION), ID 3 (1 attempt, EXITOSA), ID 4 (1 attempt, SIN_CONEXION), ID 5 (1 attempt, FALLA_NEGOCIO), ID 6 (1 attempt, SIN_CONEXION), ID 7 (1 attempt, EXITOSA), ID 8 (1 attempt, SIN_CONEXION), ID 9 (1 attempt, EXITOSA), and ID 10 (1 attempt, SIN_CONEXION).

```
PS C:\Users\padid\AppData\Local\Programs\Microsoft VS Code> & c:\Users\padid\AppData\Local\Programs\Python\Python314\python.exe c:/Users/padid/Documents/Ejercicio_Python/cola.py
s.py
Simulador de cola de transacciones
ID: 1 Intento: 1 Resultado: EXITOSA
ID: 2 Intento: 1 Resultado: SIN_CONEXION
ID: 3 Intento: 1 Resultado: EXITOSA
ID: 4 Intento: 1 Resultado: SIN_CONEXION
ID: 5 Intento: 1 Resultado: FALLA_NEGOCIO
ID: 6 Intento: 1 Resultado: SIN_CONEXION
ID: 7 Intento: 1 Resultado: EXITOSA
ID: 8 Intento: 1 Resultado: SIN_CONEXION
ID: 9 Intento: 1 Resultado: EXITOSA
ID: 10 Intento: 1 Resultado: SIN_CONEXION
```

The second screenshot shows the same script being run again, but with different results for some transactions. The results are: ID 9 (1 attempt, SIN_CONEXION), ID 10 (1 attempt, SIN_CONEXION), ID 9 (2 attempt, SIN_CONEXION), ID 9 (3 attempt, FALLA_SISTEMA), ID 10 (1 attempt, SIN_CONEXION), and ID 10 (2 attempt, EXITOSA).

```
PS C:\Users\padid\AppData\Local\Programs\Microsoft VS Code> & c:\Users\padid\AppData\Local\Programs\Python\Python314\python.exe c:/Users/padid/Documents/Ejercicio_Python/cola.py
s.py
ID: 9 Intento: 1 Resultado: SIN_CONEXION
ID: 10 Intento: 1 Resultado: SIN_CONEXION
ID: 9 Intento: 2 Resultado: SIN_CONEXION
ID: 9 Intento: 3 Resultado: FALLA_SISTEMA
ID: 10 Intento: 1 Resultado: SIN_CONEXION
ID: 10 Intento: 2 Resultado: EXITOSA
```

2. Resumen final de la ejecución del programa.



The screenshot shows a summary of the transaction simulation results. It lists the total number of transactions (10), the number of successful transactions (8), the number of failed transactions (1), the number of failed transactions due to system failure (1), the total number of retries (7), and the success rate (80.0%).

```
RESUMEN
Total de transacciones: 10
Exitosas: 8
Fallidas por negocio: 1
Fallidas por sistema: 1
Total de reintentos: 7
Porcentaje de éxito: 80.0 %
PS C:\Users\padid\AppData\Local\Programs\Microsoft VS Code> & c:\Users\padid\AppData\Local\Programs\Python\Python314\python.exe c:/Users/padid/Documents/Ejercicio_Python/cola.py
```

9. Dificultades encontradas

Durante el desarrollo, una de las dificultades fue organizar correctamente los reintentos de las transacciones que tenían problemas de conexión.

También se presentó un problema porque algunos resultados aparecían repetidos en la terminal. Al revisar el código, encontré que la función de los intentos se estaba llamando nuevamente después de haber sido ejecutada. Se eliminó esa llamada adicional y se volvió a ejecutar el programa para comprobar que cada transacción apareciera una sola vez.

Otra dificultad fue llevar correctamente el conteo de las transacciones exitosas, las fallidas y los reintentos. Para solucionarlo, se utilizaron variables que van aumentando según el resultado de cada transacción.

10. Qué mejoraría

Si tuviera más tiempo y conocimientos, mejoraría la forma de mostrar los resultados en la terminal para que fueran más fáciles de leer.

También podría agregar más validaciones para los datos de las transacciones y probar el programa con diferentes cantidades de datos.

Otra mejora sería organizar el registro de los intentos de una forma más completa para poder consultar fácilmente el resultado de cada transacción.